

LIBRARY MANAGEMENT DATABASE SYSTEM USING POSTGRESQL

SQL Query For Day1:

--create schema

CREATE SCHEMA library;

CREATE SCHEMA transactions;

--Create Authors Table

```
CREATE TABLE library.authors (  
    author_id SERIAL PRIMARY KEY,  
    author_name VARCHAR(100) NOT NULL,  
    country VARCHAR(50) NOT NULL  
);
```

--Create Books Table

```
CREATE TABLE library.books (  
    book_id SERIAL PRIMARY KEY,  
    title VARCHAR(150) NOT NULL,  
    price NUMERIC(10,2) CHECK (price > 0),  
    stock INTEGER CHECK (stock >= 0),  
    author_id INTEGER,  
  
    CONSTRAINT fk_author  
        FOREIGN KEY(author_id)
```

```
REFERENCES library.authors(author_id)

);

--Create Members Table

CREATE TABLE library.members (

    member_id SERIAL PRIMARY KEY,

    member_name VARCHAR(100) NOT NULL,

    email VARCHAR(100) UNIQUE NOT NULL,

    phone VARCHAR(15) UNIQUE

);

--Create Borrow Records Table

CREATE TABLE transactions.borrow_records (

    borrow_id SERIAL PRIMARY KEY,

    member_id INTEGER NOT NULL,

    book_id INTEGER NOT NULL,

    borrow_date DATE NOT NULL,

    return_date DATE,

    CONSTRAINT fk_member

        FOREIGN KEY(member_id)

        REFERENCES library.members(member_id),

    CONSTRAINT fk_book

        FOREIGN KEY(book_id)

        REFERENCES library.books(book_id)

);
```

--Create Fines Table

```
CREATE TABLE transactions.fines (  
    fine_id SERIAL PRIMARY KEY,  
    borrow_id INTEGER UNIQUE,  
    fine_amount NUMERIC(10,2) CHECK (fine_amount >= 0),  
  
    CONSTRAINT fk_borrow  
        FOREIGN KEY(borrow_id)  
        REFERENCES transactions.borrow_records(borrow_id)  
);
```

--Insert Authors

```
INSERT INTO library.authors(author_name, country)  
VALUES  
( 'J.K. Rowling', 'UK'),  
( 'Chetan Bhagat', 'India'),  
( 'Dan Brown', 'USA'),  
( 'Paulo Coelho', 'Brazil'),  
( 'Robin Sharma', 'Canada');
```

--Insert Books

```
INSERT INTO library.books(title, price, stock, author_id)  
VALUES  
( 'Harry Potter', 500, 10, 1),  
( 'Half Girlfriend', 300, 15, 2),  
( 'Da Vinci Code', 450, 8, 3),
```

('The Alchemist', 350, 20, 4),
('Monk Who Sold Ferrari', 400, 12, 5),
('Inferno', 550, 5, 3),

--Insert Members

INSERT INTO library.members(member_name, email, phone)

VALUES

('Arun', 'arun@gmail.com', '9876543210'),
('Kumar', 'kumar@gmail.com', '9876543211'),
('Priya', 'priya@gmail.com', '9876543212'),
('Sneha', 'sneha@gmail.com', '9876543213'),
('Rahul', 'rahul@gmail.com', '9876543214'),
('Vijay', 'vijay@gmail.com', '9876543215'),
('Ajith', 'ajith@gmail.com', '9876543216'),
('Meena', 'meena@gmail.com', '9876543217'),
('Rani', 'rani@gmail.com', '9876543218'),
('Karthik', 'karthik@gmail.com', '9876543219');

--Insert Borrow Records

INSERT INTO transactions.borrow_records

(member_id, book_id, borrow_date, return_date)

VALUES

(1,1,'2026-05-01','2026-05-10'),
(2,2,'2026-05-02','2026-05-08'),
(3,3,'2026-05-03','2026-05-11'),
(4,4,'2026-05-04','2026-05-12'),
(5,5,'2026-05-05','2026-05-15'),

```
(6,6,'2026-05-06','2026-05-14'),  
(7,7,'2026-05-07','2026-05-18'),  
(8,8,'2026-05-08','2026-05-20'),  
(9,9,'2026-05-09','2026-05-21'),  
(10,10,'2026-05-10','2026-05-22');
```

--Insert Fines

```
INSERT INTO transactions.fines(borrow_id, fine_amount)  
VALUES  
(1,50),  
(3,100),  
(5,75),  
(7,40),  
(9,60);
```

--JOIN Query

```
SELECT  
    m.member_name,  
    b.title,  
    br.borrow_date  
FROM transactions.borrow_records br  
JOIN library.members m  
ON br.member_id = m.member_id  
JOIN library.books b  
ON br.book_id = b.book_id;
```

--GROUP BY & HAVING

```
SELECT
    a.author_name,
    COUNT(b.book_id) AS total_books
FROM library.authors a
JOIN library.books b
ON a.author_id = b.author_id
GROUP BY a.author_name
HAVING COUNT(b.book_id) > 1;
```

--Subquery

```
SELECT title, price
FROM library.books
WHERE price >
(
    SELECT AVG(price)
    FROM library.books
);
```

-- CTE

```
WITH expensive_books AS (
    SELECT title, price
    FROM library.books
    WHERE price > 400
)
```

```
SELECT *
FROM expensive_books;
```

--Window Function

SELECT

title,

price,

RANK() OVER(ORDER BY price DESC) AS price_rank

FROM library.books;

--Use EXPLAIN ANALYZE

EXPLAIN ANALYZE

SELECT *

FROM library.books

WHERE price > 400;

--create view

CREATE VIEW library.book_details AS

SELECT

b.title,

a.author_name,

b.price

FROM library.books b

JOIN library.authors a

ON b.author_id = a.author_id;

--view

SELECT * FROM library.book_details;

--create materialized view

CREATE MATERIALIZED VIEW library.mv_book_details AS

SELECT

 b.title,

 a.author_name,

 b.price

FROM library.books b

JOIN library.authors a

ON b.author_id = a.author_id;

--use materialized view

SELECT * FROM library.mv_book_details;

--Refresh Materialized View

REFRESH MATERIALIZED VIEW library.mv_book_details;

--Create Function

CREATE OR REPLACE FUNCTION library.calculate_gst(price NUMERIC)

RETURNS NUMERIC AS

\$\$

BEGIN

 RETURN price + (price * 0.18);

END;

\$\$

LANGUAGE plpgsql;

--Use Function

SELECT

title,

price,

library.calculate_gst(price) AS gst_price

FROM library.books;

--Create Procedure

CREATE OR REPLACE PROCEDURE library.update_stock(

b_id INTEGER,

new_stock INTEGER

)

LANGUAGE plpgsql

AS

\$\$

BEGIN

UPDATE library.books

SET stock = new_stock

WHERE book_id = b_id;

END;

\$\$;

--Execute Procedure

CALL library.update_stock(1, 25);

Library Schema:

1.Author Table

	author_id [PK] integer	author_name character varying (100)	country character varying (50)
1	1	J.K. Rowling	UK
2	2	Chetan Bhagat	India
3	3	Dan Brown	USA
4	4	Paulo Coelho	Brazil
5	5	Robin Sharma	Canada

2.Books Table

	book_id [PK] integer	title character varying (150)	price numeric (10,2)	stock integer	author_id integer	updated_at timestamp without time zone
1	2	Half Girlfriend	300.00	15	2	[null]
2	3	Da Vinci Code	450.00	8	3	[null]
3	4	The Alchemist	350.00	20	4	[null]
4	5	Monk Who Sold Ferrari	400.00	12	5	[null]
5	6	Inferno	550.00	5	3	[null]
6	7	Revolution 2020	320.00	10	2	[null]
7	8	Ikigai	280.00	18	5	[null]
8	9	Digital Fortress	430.00	6	3	[null]
9	10	Veronika Decides to Die	390.00	7	4	[null]
10	1	Harry Potter	500.00	29	1	2026-05-29 09:23:28.316495

3.Members Table

	member_id [PK] integer	member_name character varying (100)	email character varying (100)	phone character varying (15)
1	1	Arun	arun@gmail.com	9876543210
2	2	Kumar	kumar@gmail.com	9876543211
3	3	Priya	priya@gmail.com	9876543212
4	4	Sneha	sneha@gmail.com	9876543213
5	5	Rahul	rahul@gmail.com	9876543214
6	6	Vijay	vijay@gmail.com	9876543215
7	7	Ajith	ajith@gmail.com	9876543216
8	8	Meena	meena@gmail.com	9876543217
9	9	Rani	rani@gmail.com	9876543218
10	10	Karthik	karthik@gmail.com	9876543219

Transactions Schema:

1.Borrow Records Table

	borrow_id [PK] integer	member_id integer	book_id integer	borrow_date date	return_date date
1	1	1	1	2026-05-01	2026-05-10
2	2	2	2	2026-05-02	2026-05-08
3	3	3	3	2026-05-03	2026-05-11
4	4	4	4	2026-05-04	2026-05-12
5	5	5	5	2026-05-05	2026-05-15
6	6	6	6	2026-05-06	2026-05-14
7	7	7	7	2026-05-07	2026-05-18
8	8	8	8	2026-05-08	2026-05-20
9	9	9	9	2026-05-09	2026-05-21
10	10	10	10	2026-05-10	2026-05-22

2.Fines Table

	fine_id [PK] integer 	borrow_id integer 	fine_amount numeric (10,2) 
1	1	1	50.00
2	2	3	100.00
3	3	5	75.00
4	4	7	40.00
5	5	9	60.00

Day1 Screenshots:

Joins

	member_name character varying (100) 	title character varying (150) 	borrow_date date 
1	Arun	Harry Potter	2026-05-01
2	Kumar	Half Girlfriend	2026-05-02
3	Priya	Da Vinci Code	2026-05-03
4	Sneha	The Alchemist	2026-05-04
5	Rahul	Monk Who Sold Ferrari	2026-05-05
6	Vijay	Inferno	2026-05-06
7	Ajith	Revolution 2020	2026-05-07
8	Meena	Ikigai	2026-05-08
9	Rani	Digital Fortress	2026-05-09
10	Karthik	Veronika Decides to Die	2026-05-10

Group By And Having

	author_name character varying (100) 🔒	total_books bigint 🔒
1	Chetan Bhagat	2
2	Dan Brown	3
3	Paulo Coelho	2
4	Robin Sharma	2

Subqueries

	title character varying (150) 🔒	price numeric (10,2) 🔒
1	Da Vinci Code	450.00
2	Monk Who Sold Ferrari	400.00
3	Inferno	550.00
4	Digital Fortress	430.00
5	Harry Potter	500.00

Common Table Expression

	title character varying (150) 🔒	price numeric (10,2) 🔒
1	Da Vinci Code	450.00
2	Inferno	550.00
3	Digital Fortress	430.00
4	Harry Potter	500.00

Window Functions

	title character varying (150) 🔒	price numeric (10,2) 🔒	price_rank bigint 🔒
1	Inferno	550.00	1
2	Harry Potter	500.00	2
3	Da Vinci Code	450.00	3
4	Digital Fortress	430.00	4
5	Monk Who Sold Ferrari	400.00	5
6	Veronika Decides to Die	390.00	6
7	The Alchemist	350.00	7
8	Revolution 2020	320.00	8
9	Half Girlfriend	300.00	9
10	Ikigai	280.00	10

Explain Analyze

	QUERY PLAN text 🔒
1	Seq Scan on books (cost=0.00..1.12 rows=3 width=354) (actual time=0.032..0.034 rows=4.00 loop...
2	Filter: (price > '400'::numeric)
3	Rows Removed by Filter: 6
4	Buffers: shared hit=1
5	Planning Time: 0.151 ms
6	Execution Time: 0.053 ms

View

	title character varying (150) 🔒	author_name character varying (100) 🔒	price numeric (10,2) 🔒
1	Harry Potter	J.K. Rowling	500.00
2	Revolution 2020	Chetan Bhagat	320.00
3	Half Girlfriend	Chetan Bhagat	300.00
4	Digital Fortress	Dan Brown	430.00
5	Inferno	Dan Brown	550.00
6	Da Vinci Code	Dan Brown	450.00
7	Veronika Decides to Die	Paulo Coelho	390.00
8	The Alchemist	Paulo Coelho	350.00
9	Ikigai	Robin Sharma	280.00
10	Monk Who Sold Ferrari	Robin Sharma	400.00

Materialized View

	title character varying (150) 🔒	author_name character varying (100) 🔒	price numeric (10,2) 🔒
1	Harry Potter	J.K. Rowling	500.00
2	Half Girlfriend	Chetan Bhagat	300.00
3	Da Vinci Code	Dan Brown	450.00
4	The Alchemist	Paulo Coelho	350.00
5	Monk Who Sold Ferrari	Robin Sharma	400.00
6	Inferno	Dan Brown	550.00
7	Revolution 2020	Chetan Bhagat	320.00
8	Ikigai	Robin Sharma	280.00
9	Digital Fortress	Dan Brown	430.00
10	Veronika Decides to Die	Paulo Coelho	390.00

Function

	title character varying (150) 🔒	price numeric (10,2) 🔒	gst_price numeric 🔒
1	Half Girlfriend	300.00	354.0000
2	Da Vinci Code	450.00	531.0000
3	The Alchemist	350.00	413.0000
4	Monk Who Sold Ferrari	400.00	472.0000
5	Inferno	550.00	649.0000
6	Revolution 2020	320.00	377.6000
7	Ikigai	280.00	330.4000
8	Digital Fortress	430.00	507.4000
9	Veronika Decides to Die	390.00	460.2000
10	Harry Potter	500.00	590.0000

Procedure

	book_id [PK] integer 🔗	title character varying (150) 🔗	price numeric (10,2) 🔗	stock integer 🔗	author_id integer 🔗	updated_at timestamp without time zone 🔗
1	2	Half Girlfriend	300.00	15	2	[null]
2	3	Da Vinci Code	450.00	8	3	[null]
3	4	The Alchemist	350.00	20	4	[null]
4	5	Monk Who Sold Ferrari	400.00	12	5	[null]
5	6	Inferno	550.00	5	3	[null]
6	7	Revolution 2020	320.00	10	2	[null]
7	8	Ikigai	280.00	18	5	[null]
8	9	Digital Fortress	430.00	6	3	[null]
9	10	Veronika Decides to Die	390.00	7	4	[null]
10	1	Harry Potter	500.00	25	1	2026-05-29 14:53:59.623394

Day1 Observations/Learning Notes:

1. Learned how to install PostgreSQL and connect using pgAdmin.
2. Understood how schemas help organize tables inside a database.
3. Learned how to create tables using SQL commands.
4. Understood the importance of Primary Key and Foreign Key relationships.
5. Learned different constraints such as:
 - Primary Key
 - Foreign Key
 - Unique
 - Not Null
 - Check Constraint
6. Understood how to insert sample data into tables.
7. Learned how JOIN queries combine data from multiple tables.
8. Understood Group By and Having clauses for grouped data filtering.
9. Learned how Subqueries work inside another query.
10. Understood CTE (Common Table Expression) and how it improves query readability.
11. Learned Window Functions like RANK() for row-wise calculations.
12. Understood query performance using EXPLAIN ANALYZE.
13. Learned the purpose of Views and Materialized Views.
14. Understood the difference between View and Materialized View.
15. Learned how Functions return calculated values.
16. Understood how Procedures perform database operations.

SQL Query For Day2:

--Index

```
CREATE INDEX idx_book_title  
ON library.books(title);
```

```
SELECT *  
FROM library.books  
WHERE title = 'Inferno';
```

--Create Index for High Selectivity

```
CREATE INDEX idx_member_email  
ON library.members(email);
```

```
SELECT *  
FROM library.members  
WHERE email = 'arun@gmail.com';
```

--Create Index for Low Selectivity

```
CREATE INDEX idx_author_id  
ON library.books(author_id);
```

```
SELECT *  
FROM library.books  
WHERE author_id = 3;
```

--Trigger

--Add New Column

```
ALTER TABLE library.books
```

```
ADD COLUMN updated_at TIMESTAMP;
```

```
--Create Trigger Function
```

```
CREATE OR REPLACE FUNCTION library.update_timestamp()
```

```
RETURNS TRIGGER AS
```

```
$$
```

```
BEGIN
```

```
    NEW.updated_at = CURRENT_TIMESTAMP;
```

```
    RETURN NEW;
```

```
END;
```

```
$$
```

```
LANGUAGE plpgsql;
```

```
--Create Trigger
```

```
CREATE TRIGGER trg_update_timestamp
```

```
BEFORE UPDATE
```

```
ON library.books
```

```
FOR EACH ROW
```

```
EXECUTE FUNCTION library.update_timestamp();
```

```
--Test Trigger
```

```
UPDATE library.books
```

```
SET stock = 30
```

```
WHERE book_id = 1;
```

```
SELECT *  
FROM library.books  
WHERE book_id = 1;
```

```
--Begin, Commit  
BEGIN;
```

```
UPDATE library.books  
SET stock = stock - 1  
WHERE book_id = 1;
```

```
COMMIT;
```

```
SELECT *  
FROM library.books  
WHERE book_id = 1;
```

```
--Begin, Rollback  
BEGIN;
```

```
UPDATE library.books  
SET stock = 0  
WHERE book_id = 2;
```

```
ROLLBACK;
```

```
SELECT *
```

```
FROM library.books  
WHERE book_id = 2;
```

```
--create users and provide permissions using GRANT/REVOKE  
CREATE USER library_user  
WITH PASSWORD 'library123';
```

```
GRANT SELECT, INSERT  
ON library.books  
TO library_user;
```

```
REVOKE INSERT  
ON library.books  
FROM library_user;
```

```
--Take backup and restore the database using PostgreSQL utilities
```

```
--Backup
```

```
pg_dump -U postgres -d library_db -F c -f library_db.backup
```

```
--Restore
```

```
pg_restore -U postgres -d library_restore library_db.backup
```

Day2 Screenshots:

Two Index

	book_id [PK] integer	title character varying (150)	price numeric (10,2)	stock integer	author_id integer	updated_at timestamp without time zone
1	6	Inferno	550.00	5	3	[null]

	member_id [PK] integer	member_name character varying (100)	email character varying (100)	phone character varying (15)
1	1	Arun	arun@gmail.com	9876543210

High Selectivity

	member_id [PK] integer	member_name character varying (100)	email character varying (100)	phone character varying (15)
1	1	Arun	arun@gmail.com	9876543210

Low Selectivity

	book_id [PK] integer	title character varying (150)	price numeric (10,2)	stock integer	author_id integer	updated_at timestamp without time zone
1	3	Da Vinci Code	450.00	8	3	[null]
2	6	Inferno	550.00	5	3	[null]
3	9	Digital Fortress	430.00	6	3	[null]

Trigger

	book_id [PK] integer	title character varying (150)	price numeric (10,2)	stock integer	author_id integer	updated_at timestamp without time zone
1	1	Harry Potter	500.00	25	1	2026-05-29 14:53:59.623394

Begin

	book_id [PK] integer 	title character varying (150) 	price numeric (10,2) 	stock integer 	author_id integer 	updated_at timestamp without time zone 
1	1	Harry Potter	500.00	25	1	2026-05-29 14:53:59.623394

Commit

	book_id [PK] integer 	title character varying (150) 	price numeric (10,2) 	stock integer 	author_id integer 	updated_at timestamp without time zone 
1	1	Harry Potter	500.00	25	1	2026-05-29 14:53:59.623394

Rollback

	book_id [PK] integer 	title character varying (150) 	price numeric (10,2) 	stock integer 	author_id integer 	updated_at timestamp without time zone 
1	2	Half Girlfriend	300.00	15	2	[null]

Day2 Observations/Learning Notes:

1. Learned how indexes improve query performance.
2. Understood High Selectivity and Low Selectivity concepts.
3. Learned that indexes work better on highly unique columns like email.
4. Understood how triggers automatically execute during INSERT, UPDATE, or DELETE operations.
5. Learned how transactions maintain data consistency.
6. Understood the use of BEGIN, COMMIT, and ROLLBACK.
7. Learned how to create users and roles in PostgreSQL.
8. Understood how GRANT and REVOKE manage database permissions.
9. Learned how to take database backup using pgAdmin and pg_dump.
10. Understood how to restore database using pg_restore.
11. Learned how ER diagrams visually represent table relationships.
12. Understood the complete flow of designing and managing a relational database system in PostgreSQL.

ERD DIAGRAM

